

CHAPTER 15

Google Drive 설계

클라우드 파일 저장 서비스의 핵심 구조를 탐구합니다.

Block Server, Delta Sync, Metadata DB, Notification Service 등
대규모 분산 스토리지 시스템 설계의 전 과정을 다룹니다.

Block Server

Delta Sync

Metadata DB

Long Polling

Strong Consistency

10M DAU

10GB / 파일



PRESENTED BY
이승록

문제 이해 및 범위 정의

Problem Understanding & Scope

면접관과의 대화 — 핵심 질문

- 어떤 기능을 지원해야 하나요? (파일 업로드·다운로드, 공유, 알림)
- 모바일 앱도 지원하나요? (웹 + 모바일 모두)
- 어떤 파일 형식을 지원하나요? (모든 형식 허용)
- 파일은 암호화해야 하나요? (Yes, 암호화 필요)
- 파일 크기 제한이 있나요? (최대 10GB)
- 사용자 규모는? (DAU 10M)

확정된 스펙 요약

10M

DAU (일간 활성 사용자)

10GB

파일 크기 상한

지원 플랫폼

Web

iOS

Android

핵심 제약

신뢰성(Reliability) 최우선 — 데이터 유실 금지

요구사항 분석

Functional / Non-functional Requirements

기능 요구사항 (Functional)

- 파일 추가 — 드래그&드롭 방식 지원
- 파일 다운로드
- 여러 기기 간 파일 동기화 (Multi-device Sync)
- 파일 수정 이력 조회 (Revision History)
- 파일 공유 (File Sharing)
- 파일 변경 시 알림 수신 (Notification)

비기능 요구사항 (Non-functional)

- 신뢰성(Reliability) — 데이터 유실 절대 금지
- 빠른 동기화 속도 — 느린 동기화는 사용성 저하
- 대역폭 절약 — 불필요한 네트워크 사용 최소화
- 확장성(Scalability) — 대용량 트래픽 처리
- 고가용성(High Availability) — 일부 장애에도 서비스 유지

범위 외 (Out of Scope)

구글 문서(Google Docs) 실시간 공동 편집 기능 — 이 설계에서는 다루지 않습니다.

API 설계

API Design — 3가지 핵심 엔드포인트

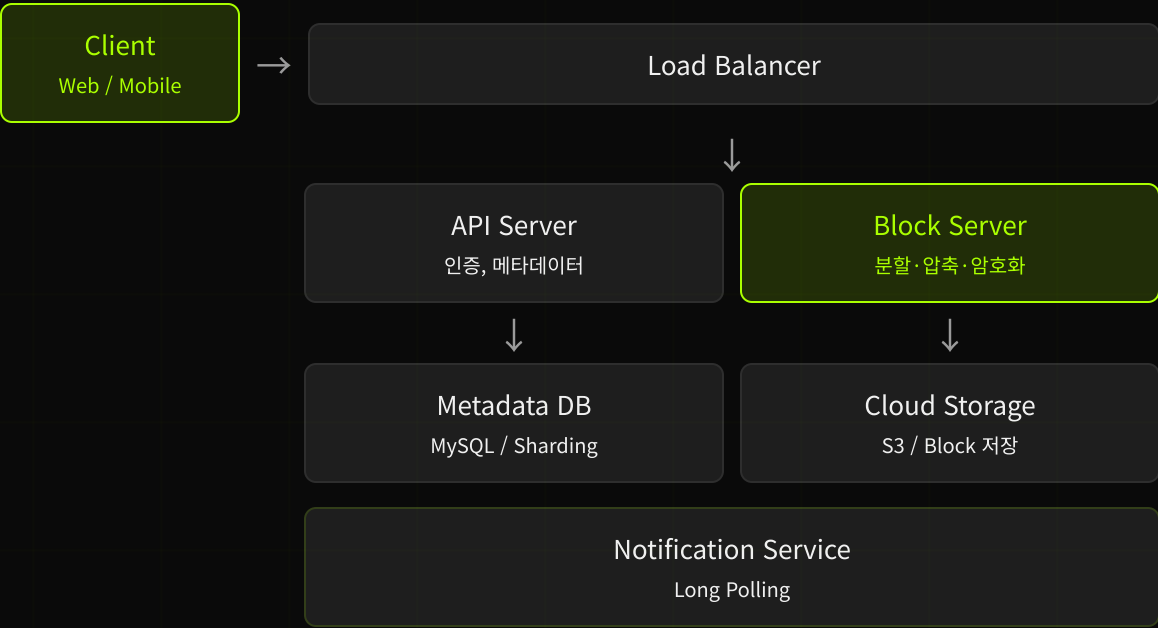
POST /files/upload		파일 업로드 (단순 / 재개 가능 방식)
파라미터	설명	
uploadType=media	단순 업로드 — 작은 파일 (5MB 이하)	
uploadType=resumable	재개 가능 업로드 — 대용량 파일, 불안정 네트워크 환경	
GET /files/download		파일 다운로드
파라미터:	path — 다운로드할 파일 경로	
GET /files/list_revisions		수정 이력 조회
파라미터:	path, limit — 조회할 파일 경로, 최대 반환 수	

모든 API는 사용자 인증(Authentication)이 필요하며, 전송 중 데이터 보호를 위해 **HTTPS / SSL**을 사용합니다.

개략적 설계

High-Level Design

주요 컴포넌트



컴포넌트 역할

컴포넌트	역할
Block Server	파일을 블록(Block) 단위로 분할, 압축(Compression), 암호화(Encryption) 후 클라우드 저장소에 업로드
Metadata DB	사용자, 파일, 블록, 버전 정보를 관리. 강한 일관성(Strong Consistency) 유지
Cloud Storage	실제 파일 블록을 저장하는 객체 스토리지 (예: AWS S3)
Notification Service	다른 클라이언트에게 파일 변경 사실을 Long Polling으로 전달

블록 서버 & Delta Sync

Block Server & Delta Synchronization

블록 서버(Block Server) 처리 흐름

1 파일을 고정 크기 **블록(Block)** 으로 분할
각 블록 크기 = 최대 4MB



2 각 블록을 **압축(Compression)**



3 각 블록을 **암호화(Encryption)**



4 클라우드 스토리지에 업로드 — 각 블록은 독립 객체

Delta Sync — 효율적인 동기화

파일의 **일부만 수정**되었을 때, 변경된 블록만 전송

Block 1
변경 없음

Block 2
수정됨 — 전송

Block 3
변경 없음

예시: 50MB 문서에서 문단 하나만 수정

기존: 50MB 전송

Delta Sync: **수 KB만 전송** — 대역폭 99% 절감

각 블록은 고유한 **해시값(Hash)** 으로 식별 — 중복 제거(De-duplication)에도 활용

메타데이터 데이터베이스

Metadata Database

저장 데이터 종류

- **사용자(User)** — 계정 정보, 인증
- **파일(File)** — 이름, 경로, 크기, 소유자, 생성/수정 시각
- **블록(Block)** — 블록 해시, 크기, 순서 정보
- **버전(Version)** — 파일 수정 이력, 이전 버전 복원 지원
- **워크스페이스(Workspace)** — 공유 디렉토리 정보

일관성 수준

Strong Consistency — 메타데이터와 캐시 레이어 모두에서 강한 일관성 유지

확장성 설계

단일 서버 문제

메타데이터 DB를 서버에 내장하면 SPOF(Single Point of Failure) 발생

해결책

- 메타데이터 DB를 **별도 서버로 분리**
- 가용성을 위한 **Replication** 설정
- 성능·확장을 위한 **Sharding** 적용
- 빠른 읽기를 위한 **Cache (Redis 등)** 레이어 추가

DB 선택: **MySQL** 계열 — 강한 일관성 지원, ACID 보장

동기화 충돌

Sync Conflict Resolution

충돌 발생 시나리오

두 사용자가 동시에 같은 파일을 수정하는 경우



핵심 원칙

먼저 처리된 버전이 원본(Latest)으로 채택

충돌 해결 전략

나중 업로드 측 처리

- 먼저 저장된 버전 = **현재 버전(Latest)**
- 나중에 저장 시도한 버전 = **충돌 사본(Conflict Copy)** 으로 저장
- 사용자가 직접 두 버전을 비교·병합

실제 결과물 예시

`report.docx` ← 원본 (User 1 버전)

`report (User 2의 충돌 사본).docx`

Dropbox, Google Drive 모두 이 방식 채택 — 자동 병합보다 **사용자 판단에 위임**하는 것이 안전

업로드 / 다운로드 흐름

Upload & Download Flow

파일 업로드 흐름 (Upload Flow)

1 클라이언트 → API 서버로 메타데이터 추가 요청
파일명, 크기, 경로 등 상태: **pending**



2 클라이언트 → Block Server로 파일 업로드
블록 분할 → 압축 → 암호화 → Cloud Storage 업로드



3 업로드 완료 → 메타데이터 상태를 **uploaded** 로 갱신



4 Notification Service → 다른 클라이언트에게 변경 알림

파일 다운로드 흐름 (Download Flow)

1 다른 클라이언트가 파일 변경 감지
Long Polling 또는 알림 수신



2 API 서버에서 최신 블록 목록(Metadata) 조회



3 Cloud Storage에서 **변경된 블록만** 다운로드
Delta Sync 적용 — 전체 파일 재다운로드 불필요



4 블록 복호화·압축 해제 → 로컬 파일 재조합

메타데이터 DB 스키마

Metadata Database Schema

user

컬럼	타입
user_id	PK
username	varchar
email	varchar
profile_photo_path	varchar

file_metadata

컬럼	타입
file_id	PK
file_name	varchar
file_size	bigint
extension	varchar
is_directory	boolean
last_modified_at	datetime
created_at	datetime
created_by	FK → user
last_modified_by	FK → user
parent_id	FK → file

block

컬럼	타입
block_id	PK
block_hash	varchar
block_size	bigint

file_version

컬럼	타입
version_id	PK
file_id	FK → file
block_id_list	순서 정보
created_time	datetime

알림 서비스 (Notification Service)

파일 변경을 다른 클라이언트에게 전달하는 메커니즘

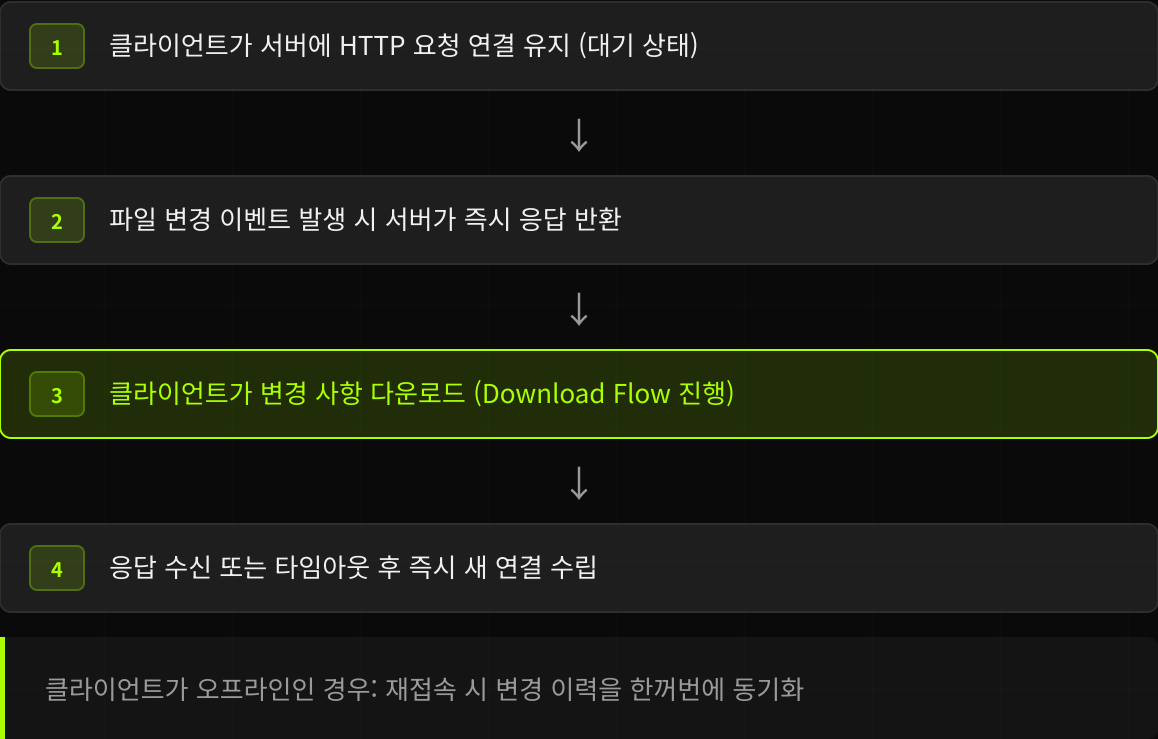
옵션 비교

방식	장점	단점
Long Polling	구현 단순, HTTP 기반	서버가 응답 후 연결 재수립 필요
WebSocket	양방향 실시간 통신	드라이브는 단방향 알림으로 충분, 오버스펙

선택: Long Polling

Google Drive는 클라이언트 → 서버 방향 알림이 주목적.
Long Polling으로 충분하며 구현이 단순함.

Long Polling 동작 방식



저장소 최적화 & 장애 처리

Storage Optimization & Failure Handling

저장소 최적화 전략

De-duplication (중복 제거)

같은 해시(Hash)를 가진 블록은 한 번만 저장.
여러 사용자가 동일 파일 업로드 시 스토리지 절감

지능적 백업 전략

버전 수 상한 설정 — 너무 오래된 버전은 자동 삭제.
중요 버전만 선택적으로 보관

Cold Storage 활용

오래된 / 거의 접근하지 않는 데이터는
저비용 **Cold Storage** (예: AWS Glacier)로 이동

장애 처리 (Failure Handling)

컴포넌트	장애 시 대응
Load Balancer	Primary 장애 시 Secondary로 자동 전환 (Active-Passive)
Block Server	무상태(Stateless) 설계 — 다른 인스턴스가 즉시 대체
Cloud Storage	다중 리전(Multi-region) Replication으로 가용성 확보
Metadata DB	Primary-Secondary Replication; Primary 장애 시 Secondary 승격
Notification Service	서버 장애 시 클라이언트는 다른 서버로 Long Polling 재연결

마무리 — Summary

핵심 개념 총정리

설계 핵심 결정 사항

주제	결정
파일 저장 방식	Block 단위 분할 저장
동기화 효율화	Delta Sync — 변경된 블록만 전송
메타데이터 DB	MySQL + Sharding + Strong Consistency
알림 방식	Long Polling
충돌 해결	나중 버전을 Conflict Copy로 보존
저장소 절감	De-duplication + Cold Storage
고가용성	Multi-region Replication + Stateless Server

기억해야 할 핵심 용어

- Block Server
- Delta Sync
- De-duplication
- Strong Consistency
- Long Polling
- Resumable Upload
- Cold Storage
- Sharding
- Sync Conflict
- SPOF

면접 포인트 요약

- 큰 파일 업로드는 **Resumable Upload** 사용 이유 설명
- Block 분할 + Hash가 De-dup과 Delta Sync 두 가지를 동시 해결
- Metadata DB는 반드시 서버에서 분리 → SPOF 제거
- 오프라인 클라이언트 재접속 시 전체 변경 이력 동기화 방법

더 생각해보기

설계를 더 깊이 파고들 질문들

Q1. 왜 WebSocket이 아닌 Long Polling인가?

Google Drive의 알림은 **단방향**(서버 → 클라이언트)으로 충분합니다. WebSocket은 양방향 채널 유지 비용이 발생합니다. 만약 실시간 공동 편집을 지원한다면 어떤 방식이 더 적합할까요?

Q2. 메타데이터 DB로 NoSQL이 아닌 RDBMS를 선택한 이유는?

파일·버전·블록 간 **관계 무결성**과 **Strong Consistency**가 핵심입니다. NoSQL(예: DynamoDB)을 선택한다면 어떤 트레이드오프가 생길까요? 특히 버전 롤백 기능 구현 시 어떤 차이가 있을까요?

Q3. 클라이언트 사이드 암호화 vs 서버 사이드 암호화

현재 설계는 **Block Server에서 암호화**합니다. 클라이언트가 직접 암호화 후 업로드한다면 보안과 기능성(예: 서버 측 검색, 썸네일 생성) 중 무엇을 잃게 될까요?

Q4. 10억 사용자로 확장 시 달라지는 것은?

현재 설계는 DAU 10M 기준입니다. DAU 1B 규모에서 Metadata DB Sharding 전략, Block Server 수평 확장, 그리고 **지역 분산(Geo-distributed)** 스토리지가 어떻게 바뀌어야 할까요?

Q5. CDN 도입 시 무엇이 달라지나?

자주 다운로드되는 파일 블록을 **CDN에 캐싱**하면 다운로드 속도를 높일 수 있습니다. 그러나 Block 단위 캐싱에서 캐시 무효화(Cache Invalidation)는 파일 단위와 어떻게 다르게 동작할까요?

Q6. 버전 관리에서 스토리지 비용을 어떻게 줄일 수 있나?

현재 버전마다 블록 목록을 저장합니다. **증분 백업(Incremental Backup)** 또는 **역방향 델타(Reverse Delta)** 방식으로 최신 버전은 전체 블록, 이전 버전은 차이(diff)만 보관하면 어떨까요?